

02 矩阵

2.1 矩阵概述

二维数组里的行、列、索引与张量起点



鸢尾花书·《线性代数不难》 | 开源代码与书稿: <https://github.com/Visualize-ML>

本节你将掌握的核心技能

1

理解矩阵的本质：按行、列排列的二维数组，既能存储数据，也能表达线性变换

3

熟练使用 NumPy 提取矩阵的行向量、列向量与单个元素，理解分块矩阵的基本思想

5

掌握矩阵加减法：两个形状相同的矩阵逐元素相加或相减，满足交换律与结合律

7

从张量视角贯通全局：标量是0阶张量、向量是1阶、矩阵是2阶，彩色图像、视频等则是更高阶张量

2

掌握行、列索引规则：行索引从上到下递增、列索引从左到右递增；数学计数从1开始，NumPy 索引从0开始

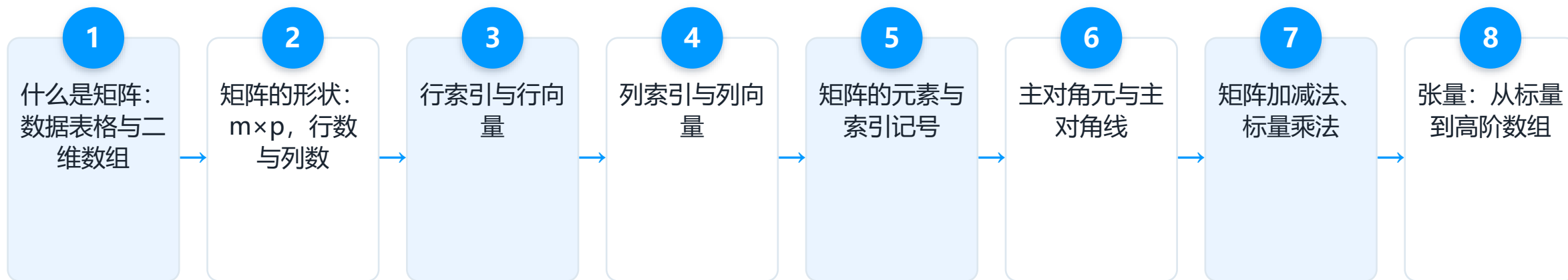
4

理解主对角元、主对角线的定义：行索引与列索引相同的一组特殊元素

6

掌握矩阵的标量乘法：用一个标量缩放矩阵中的每一个元素

本节知识脉络



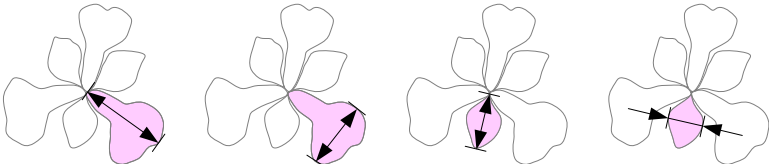
什么是矩阵？

矩阵 (matrix) 是一个按照行、列排列的二维数组

- 形象来说，矩阵就是一个充满数字的表格；比如一张学生成绩单，每一行代表一个学生，每一列代表一个科目
- 矩阵不仅是数学概念，还广泛出现在日常数据中：图片、表格、传感器读数，都能用矩阵表示
- 本节先从鸢尾花数据集、照片像素两个例子，具体看看矩阵是如何组织数据的

图1 • 鸢尾花数据集也是一个矩阵

- 每一行是一朵鸢尾花样本，前4列为花萼、花瓣的长与宽，最后一列为品种标签
- 共150个样本、3个品种；去掉表头后，就是一个 150×4 的数据矩阵



Index	Sepal length X_1	Sepal width X_2	Petal length X_3	Petal width X_4	Species C
1	5.1	3.5	1.4	0.2	Setosa C_1
2	4.9	3	1.4	0.2	
3	4.7	3.2	1.3	0.2	
...	...	...	...	...	
49	5.3	3.7	1.5	0.2	
50	5	3.3	1.4	0.2	Versicolor C_2
51	7	3.2	4.7	1.4	
52	6.4	3.2	4.5	1.5	
53	6.9	3.1	4.9	1.5	
...	...	...	...	...	
99	5.1	2.5	3	1.1	Virginica C_3
100	5.7	2.8	4.1	1.3	
101	6.3	3.3	6	2.5	
102	5.8	2.7	5.1	1.9	
103	7.1	3	5.9	2.1	
...	...	...	...	...	
149	6.2	3.4	5.4	2.3	
150	5.9	3	5.1	1.8	

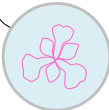


图2 • 照片本质上是一个数据矩阵

- 黑白照片可看作灰度像素组成的热图，每个像素对应矩阵中的一个元素
- 不断放大会看到离散的像素格子；提取4个像素即得到一个 2×2 矩阵

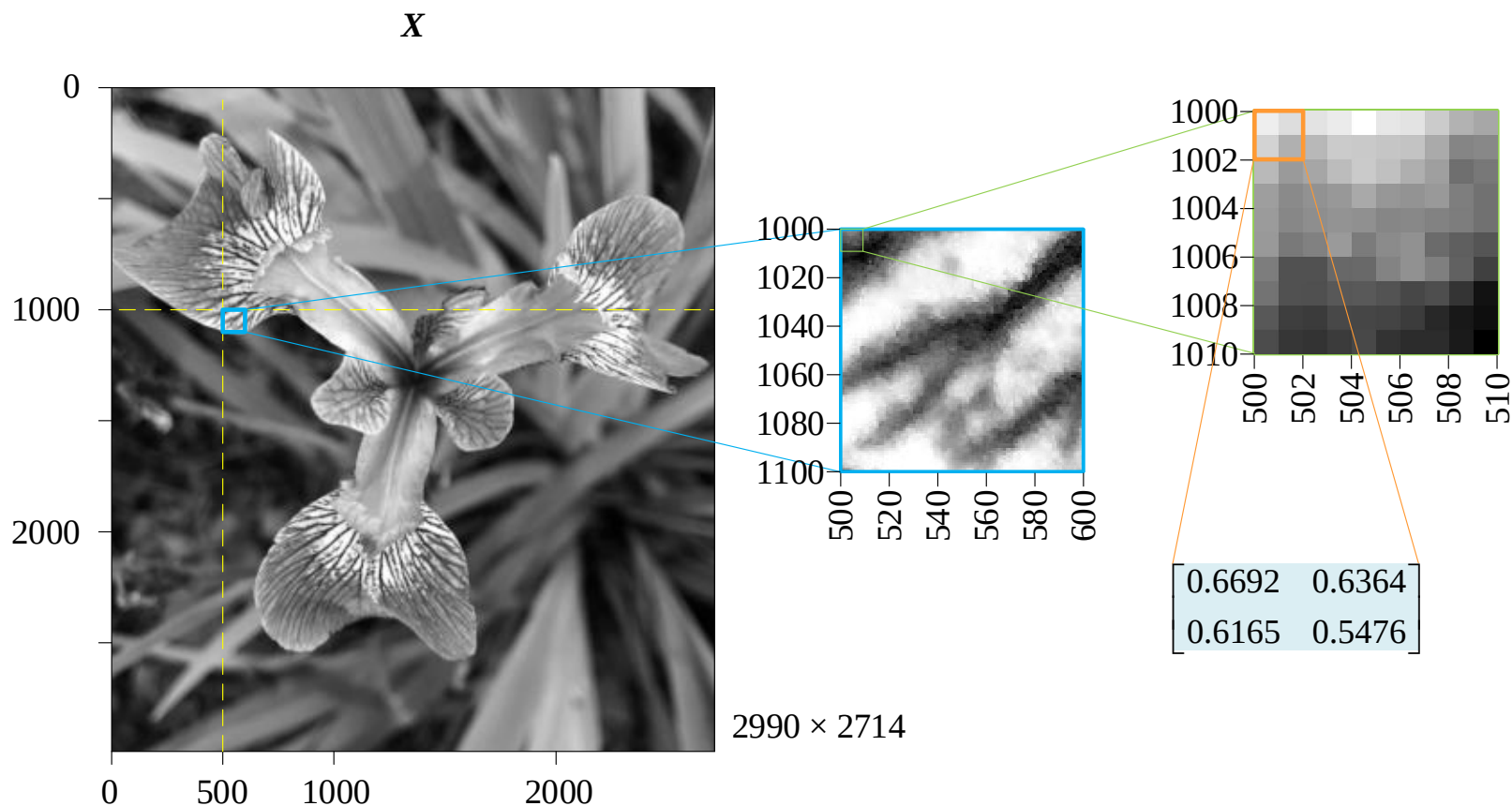
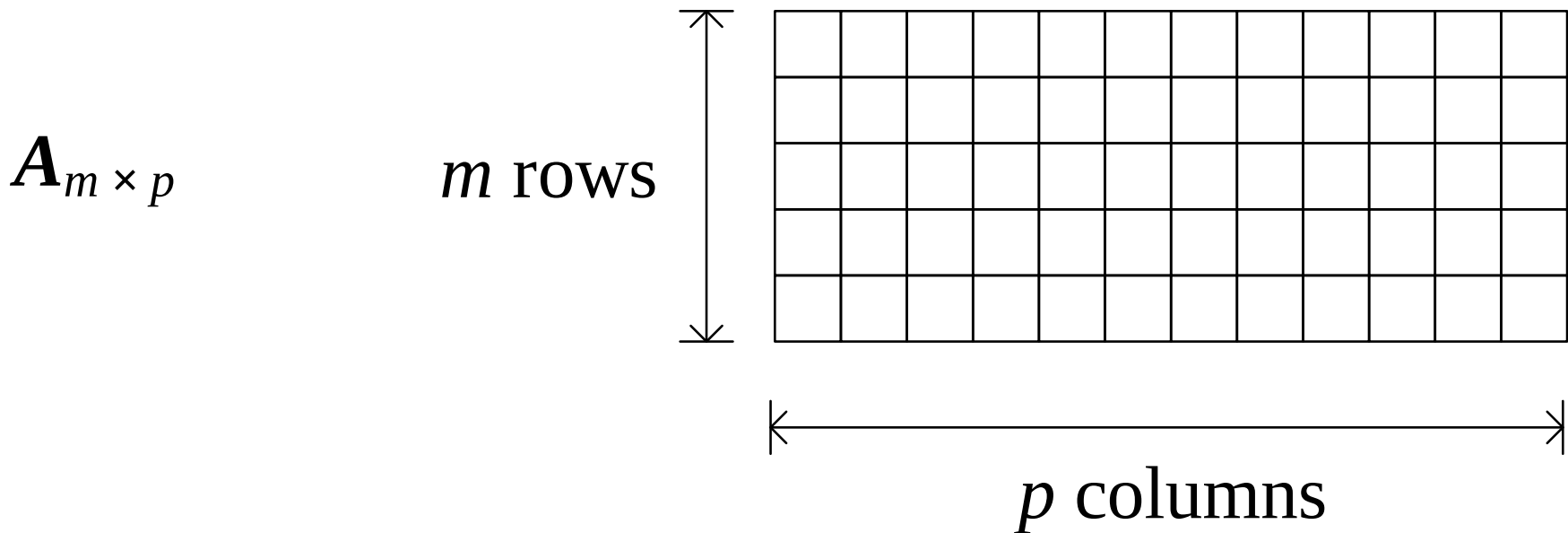


图3 • 矩阵 A 的形状记作 $m \times p$

- m 为行数, p 为列数; 矩阵记作 $A_{m \times p}$, $m \times p$ 即元素总数
- 本书中, 矩阵的每个元素都用一个方方正正的格子可视化呈现



矩阵的“生长方向”：先行后列

$A_{2 \times 3}$ 像 2 层楼、每层 3 间房； $B_{3 \times 2}$ 像 3 层楼、每层 2 间房

- 标记矩阵形状时，先写行数、再写列数——这种约定暗示了矩阵的“生长方向”
- 行向量 (row vector) 是只有一行、多列的矩阵；列向量 (column vector) 是多行、只有一列的矩阵，二者都是矩阵的特殊形状
- 方阵 (square matrix) 是行数、列数相同的矩阵

图4 • 矩阵 A 的行索引

- 行索引 i 从上到下递增, 取值1到 m
- 矩阵 A 的第 i 行记作 $\mathbf{a}^{(i)}$, 如第1行 $\mathbf{a}^{(1)}$ 、第2行 $\mathbf{a}^{(2)}$

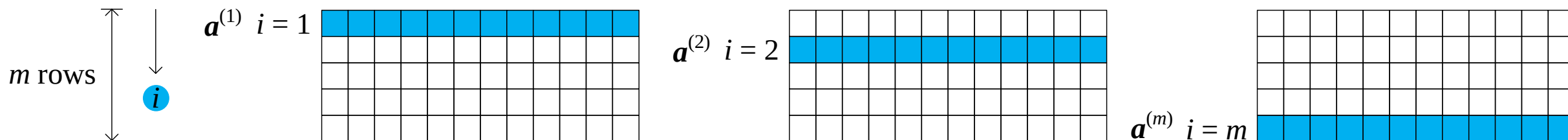
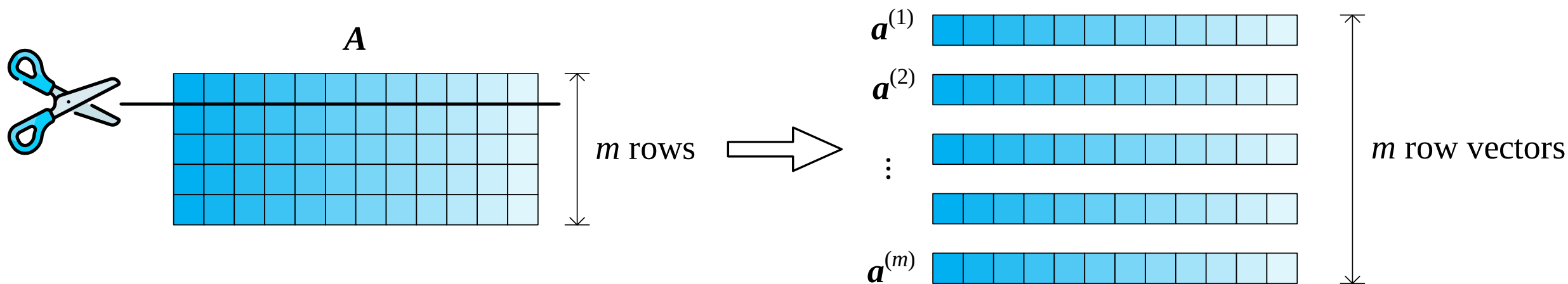


图5 · 把矩阵 A 切成一组行向量

- 矩阵可看作一组有序行向量：每一行都是一个行向量 $\mathbf{a}^{(1)}, \mathbf{a}^{(2)}, \dots, \mathbf{a}^{(m)}$
- 这正是分块矩阵 (block matrix) 思想：把大矩阵按行、列拆分为多个子矩阵 (块)



举例：把矩阵切成一组行向量

$$\mathbf{A} = [1, 2, 3; 4, 5, 6] = [\mathbf{a}^{(1)}; \mathbf{a}^{(2)}]$$

- 2×3 矩阵 $\mathbf{A}$ 的两行可以分别写成行向量 $\mathbf{a}^{(1)} = [1, 2, 3]$ 、 $\mathbf{a}^{(2)} = [4, 5, 6]$
- 同理， 3×2 矩阵 $\mathbf{B} = [1, 4; 2, 5; 3, 6]$ 可以写成三个行向量 $\mathbf{b}^{(1)} = [1, 4]$ 、 $\mathbf{b}^{(2)} = [2, 5]$ 、 $\mathbf{b}^{(3)} = [3, 6]$

NumPy 提取矩阵的行向量

用 $A[[0],:]$ 与 $A[0,:]$ 提取行, 注意维度差异

- `np.random.seed(88)` 固定随机种子, `np.random.randint(0,10,(5,12))` 生成一个 5×12 的随机整数矩阵
- `seaborn.heatmap()` 把矩阵可视化为主图, `cmap='RdYlBu_r'` 设置红黄蓝渐变配色, `square=True` 保持单元格为正方形
- `A[[0],:]` 返回形状为 $(1,12)$ 的二维数组, 仍保留“矩阵”的结构; `A[0,:]` 返回形状为 $(12,)$ 的一维数组
- 用负索引 `A[-1,:]` 可直接提取最后一行, 无需事先知道矩阵的具体行数

```
## 初始化
import numpy as np
import seaborn as sns

## 创建矩阵
a np.random.seed(88)
b A = np.random.randint(0, 10, (5, 12))

## 可视化
c sns.heatmap(A, cmap = 'RdYlBu_r', square = True,
               linecolor = 'w', linewidths = 0.25)

## 行索引
d a_row_1 = A[[0],:]
e A[0,:]

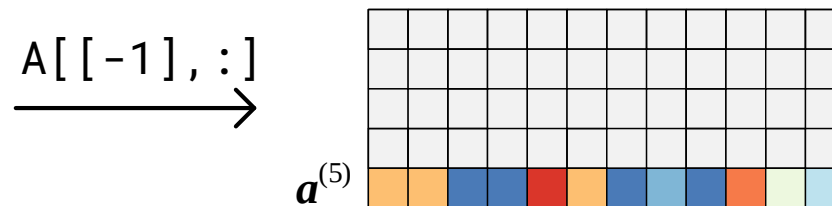
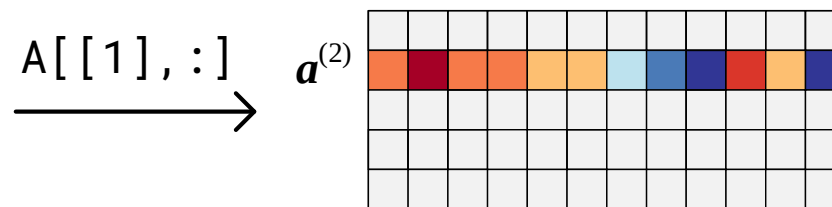
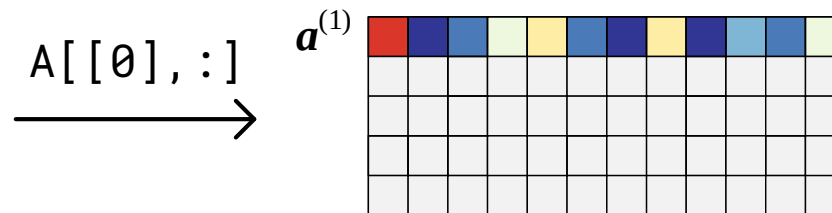
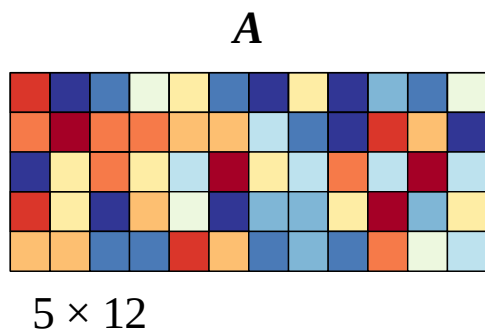
f a_row_2 = A[[1],:]

g a_row_5 = A[[-1],:]
```



图6 • 取出行向量

- $A[[0],:]$ 取第1行 $\mathbf{a}^{(1)}$; $A[[1],:]$ 取第2行 $\mathbf{a}^{(2)}$
- $A[[-1],:]$ 用负索引直接取出最后一行 $\mathbf{a}^{(5)}$



列

图7 • 矩阵 A 的列索引

- 列索引 k 从左到右递增, 取值1到 p
- 矩阵 A 的第 k 列记作 $\mathbf{a}_k$, 如第1列 $\mathbf{a}_1$ 、第2列 $\mathbf{a}_2$

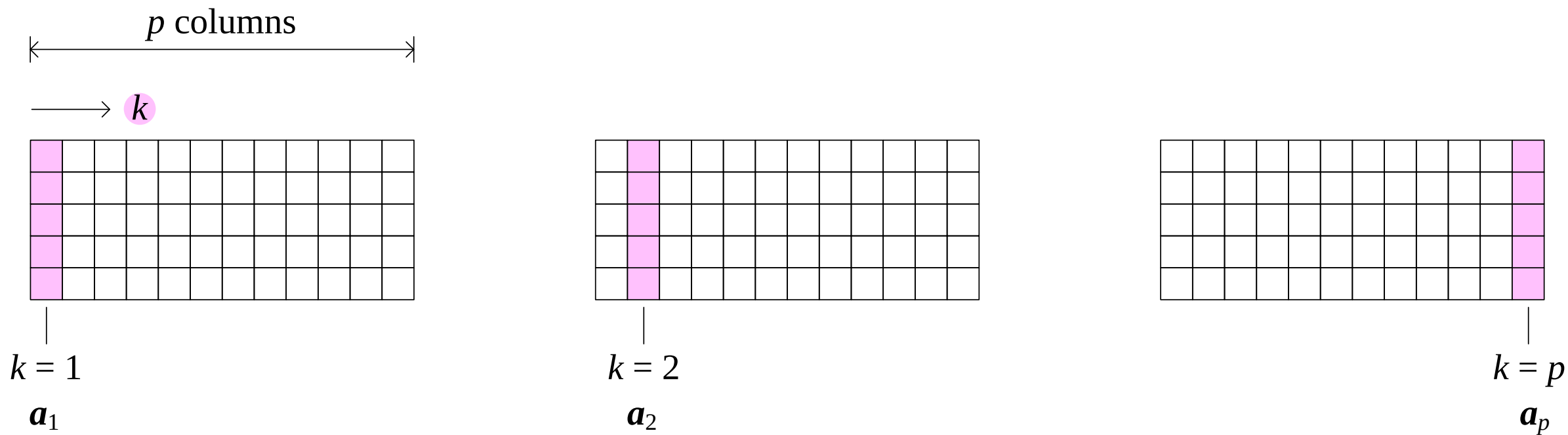
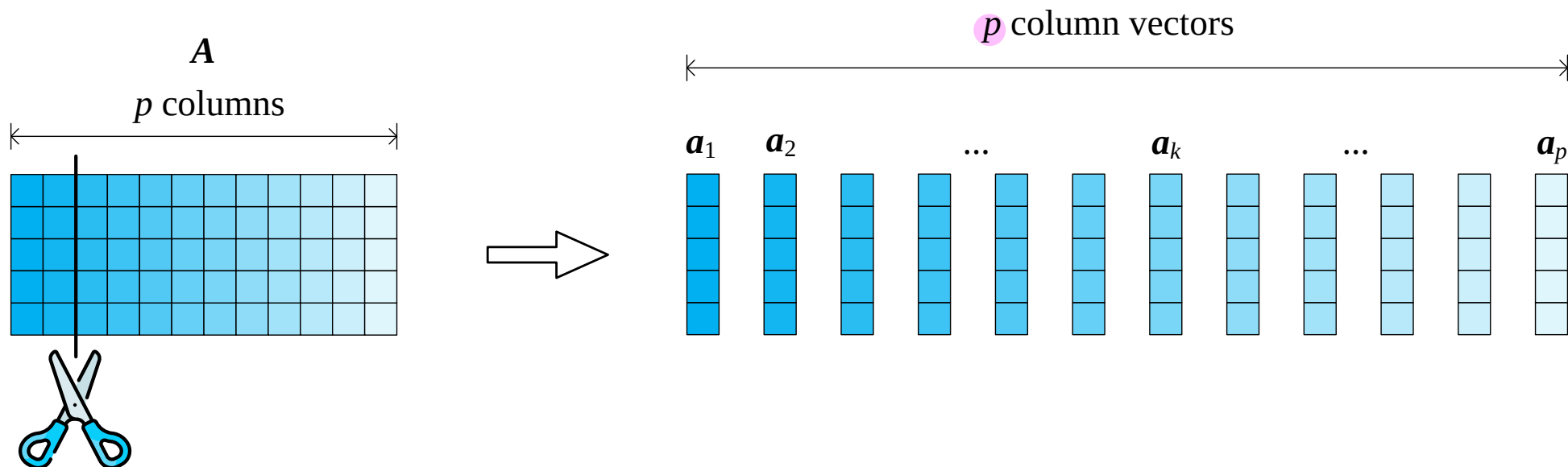


图8 · 把矩阵 A 切成一组列向量

- 矩阵也可看作一组有序列向量：每一列都是一个列向量 $\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_p$
- 这同样是分块矩阵思想，只是这次按列、而非按行切分



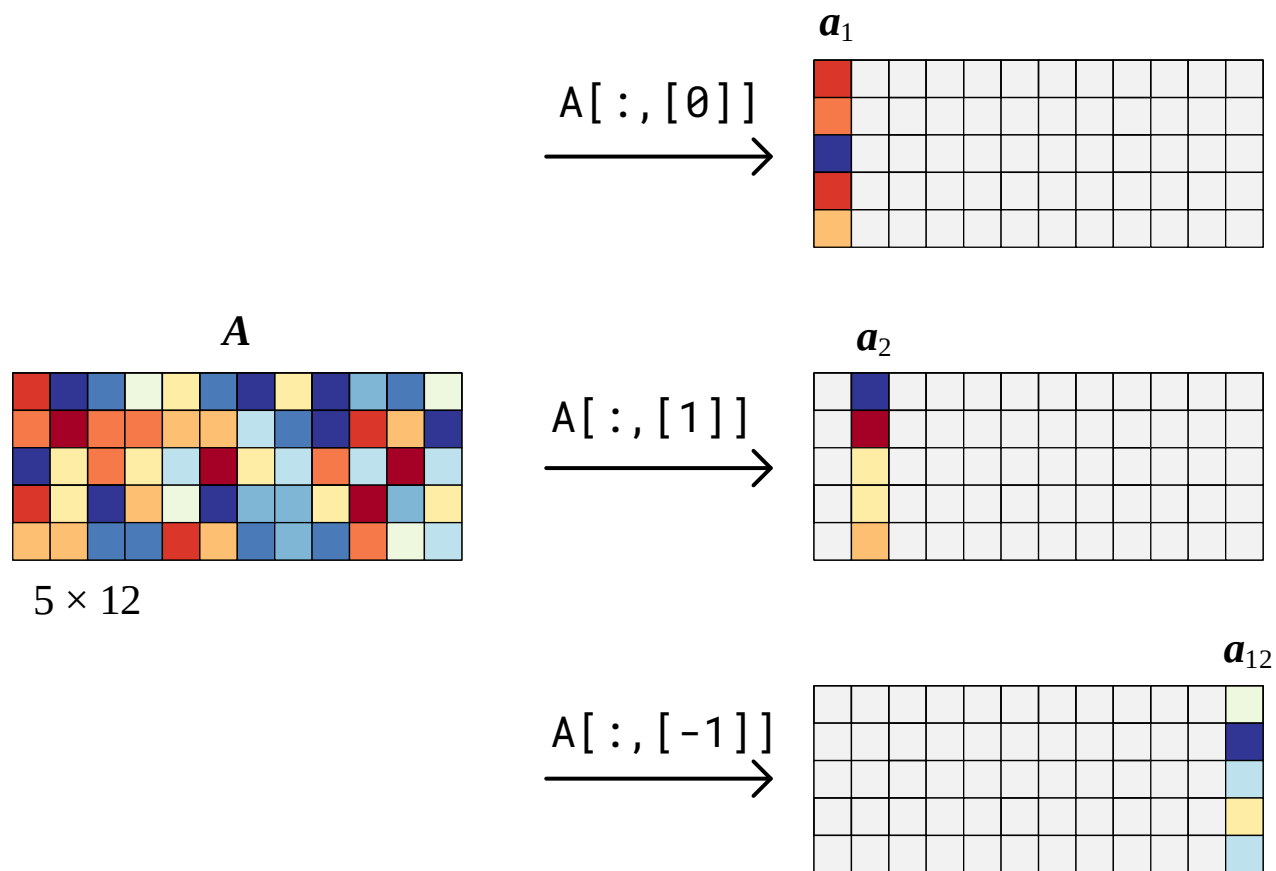
举例：把矩阵切成一组列向量

$$\mathbf{A} = [1, 2, 3; 4, 5, 6] = [\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3]$$

- 2×3 矩阵 $\mathbf{A}$ 的三列可以分别写成列向量 $\mathbf{a}_1 = [1, 4]^\top$ 、 $\mathbf{a}_2 = [2, 5]^\top$ 、 $\mathbf{a}_3 = [3, 6]^\top$
- 同理， 3×2 矩阵 $\mathbf{B} = [1, 4; 2, 5; 3, 6]$ 可以写成两个列向量 $\mathbf{b}_1 = [1, 2, 3]^\top$ 、 $\mathbf{b}_2 = [4, 5, 6]^\top$

图9 • 取出列向量

- $A[:, [0]]$ 取第1列 $\mathbf{a}_1$; $A[:, [1]]$ 取第2列 $\mathbf{a}_2$
- $A[:, [-1]]$ 用负索引取出最后一列 $\mathbf{a}_{12}$



矩阵元素的记号: $a_{i,k}$

A 第 i 行、第 k 列的元素记作 $a_{i,k}$

- 有了行、列索引, 就可以轻松标记矩阵中任意一个元素的位置
- 举例: 矩阵 A 第1行、第1列的元素记作 $a_{1,1}$; 第2行、第3列的元素记作 $a_{2,3}$

图10 • 取出元素

- $A[0,0]$ 取出元素 $a_{1,1}$; $A[2,7]$ 取出元素 $a_{3,8}$
- $A[-1,-1]$ 用负索引取出最后一个元素 $a_{5,12}$

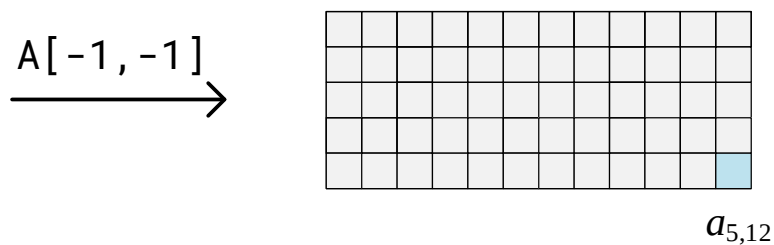
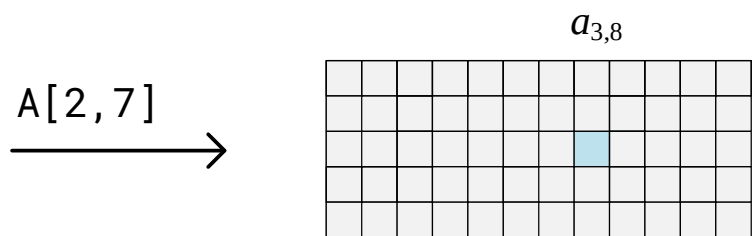
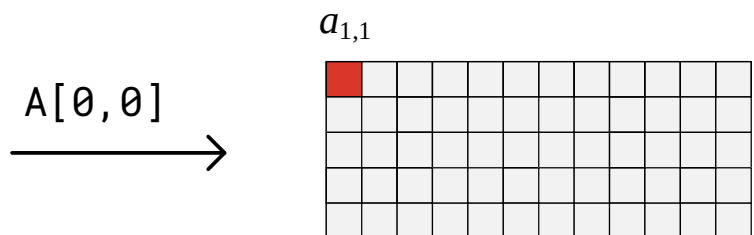
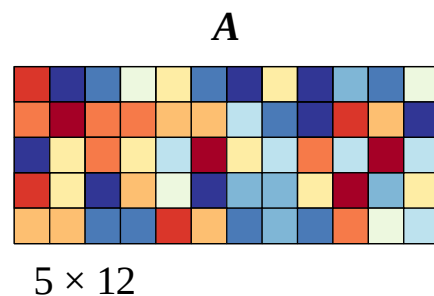


图11 • 矩阵 A 的主对角元

- 主对角元 (diagonal element) 是行、列索引相同的一组特殊元素, 如图中蓝色格子 $a_{1,1}$ 、 $a_{2,2}$ 、 $a_{3,3}$
- 主对角元的连线叫做主对角线 (main diagonal), 即图中的红线

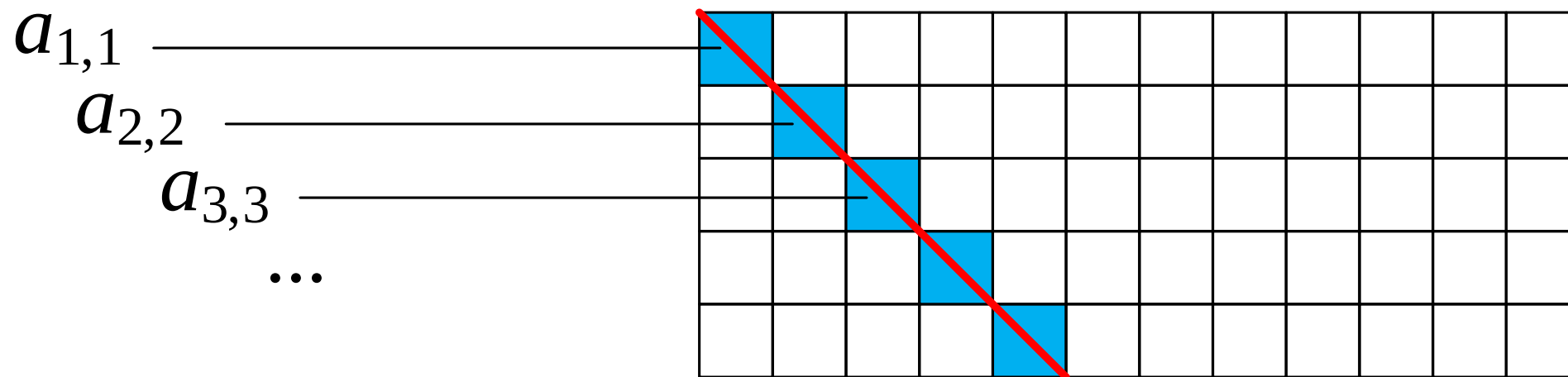


图12 • 无论形状如何变化，主对角线始终存在

- 主对角线是从矩阵左上角出发、以 45° 倾角向右下方延伸的线段
- 细高、扁平、方阵、行向量、列向量——无论形状如何，主对角线始终存在

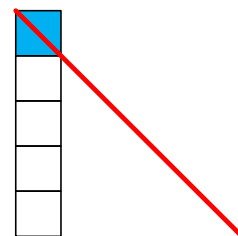
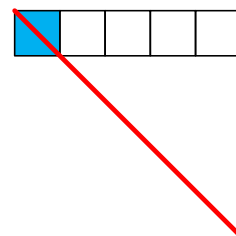
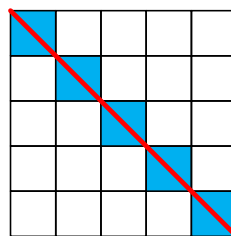
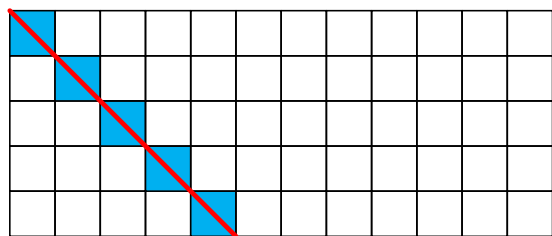
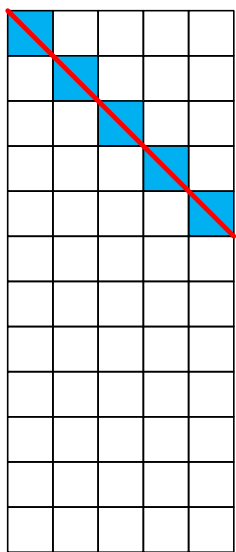
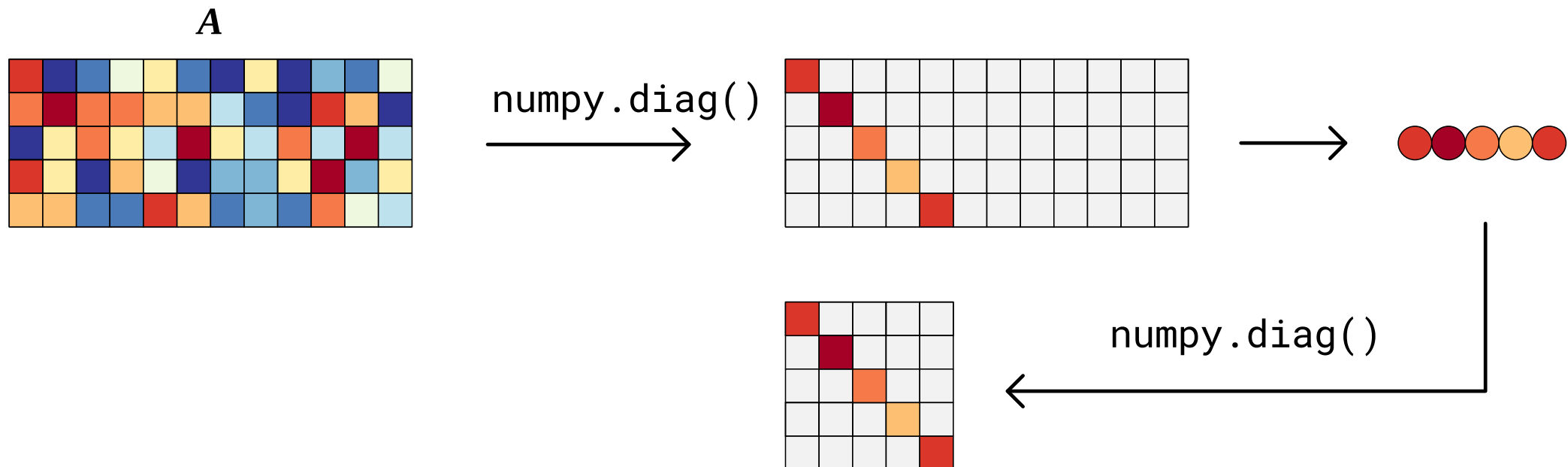


图13 • 用 `numpy.diag()` 提取或创建对角线

- `numpy.diag(A)` 提取矩阵 `A` 的主对角线元素，结果为一维数组
- 反过来，传入一维数组会创建以它为对角线、其余为零的对角方阵



两个形状相同的矩阵，可以逐元素相加减

$$(A \pm B)_{ij} = a_{ij} \pm b_{ij}$$

- 两个形状相同的矩阵 A 、 B 相加（或相减），就是把它们对应位置的元素逐一相加（或相减）
- 只有形状完全相同的矩阵才能相加减；结果矩阵的形状与原矩阵相同

矩阵加法满足交换律与结合律

$$\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$$

- 交换律 (commutative property) : 无论先加哪个矩阵, 结果始终相同
- 结合律 (associative property) : $\mathbf{A} + \mathbf{B} + \mathbf{C} = \mathbf{A} + (\mathbf{B} + \mathbf{C}) = (\mathbf{A} + \mathbf{B}) + \mathbf{C}$, 先对任意两个矩阵求和、再与第三个相加, 结果始终相同
- 上述规则同样适用于矩阵减法

零矩阵 O ：元素全为0的矩阵

$$A + O = O + A = A, \quad A - A = O$$

- 本书用 O 表示元素全为0的矩阵，即零矩阵（zero matrix）
- 任意矩阵与形状相同的零矩阵相加，结果仍然是原矩阵；矩阵减去自身，结果为零矩阵

标量乘法：用一个数缩放矩阵的每个元素

$$kA = [k \cdot a_{ij}]$$

- 矩阵的标量乘法 (scalar multiplication) 指的是用一个标量 k 乘以矩阵中的每一个元素
- 矩阵 A 的形状为 $m \times p$, 标量乘法的结果形状不变, 仍是 $m \times p$; 结果中的每个元素都被放大或缩小
- 特别地, 当 $k=0$ 时, 结果为零矩阵 O , 形状仍为 $m \times p$

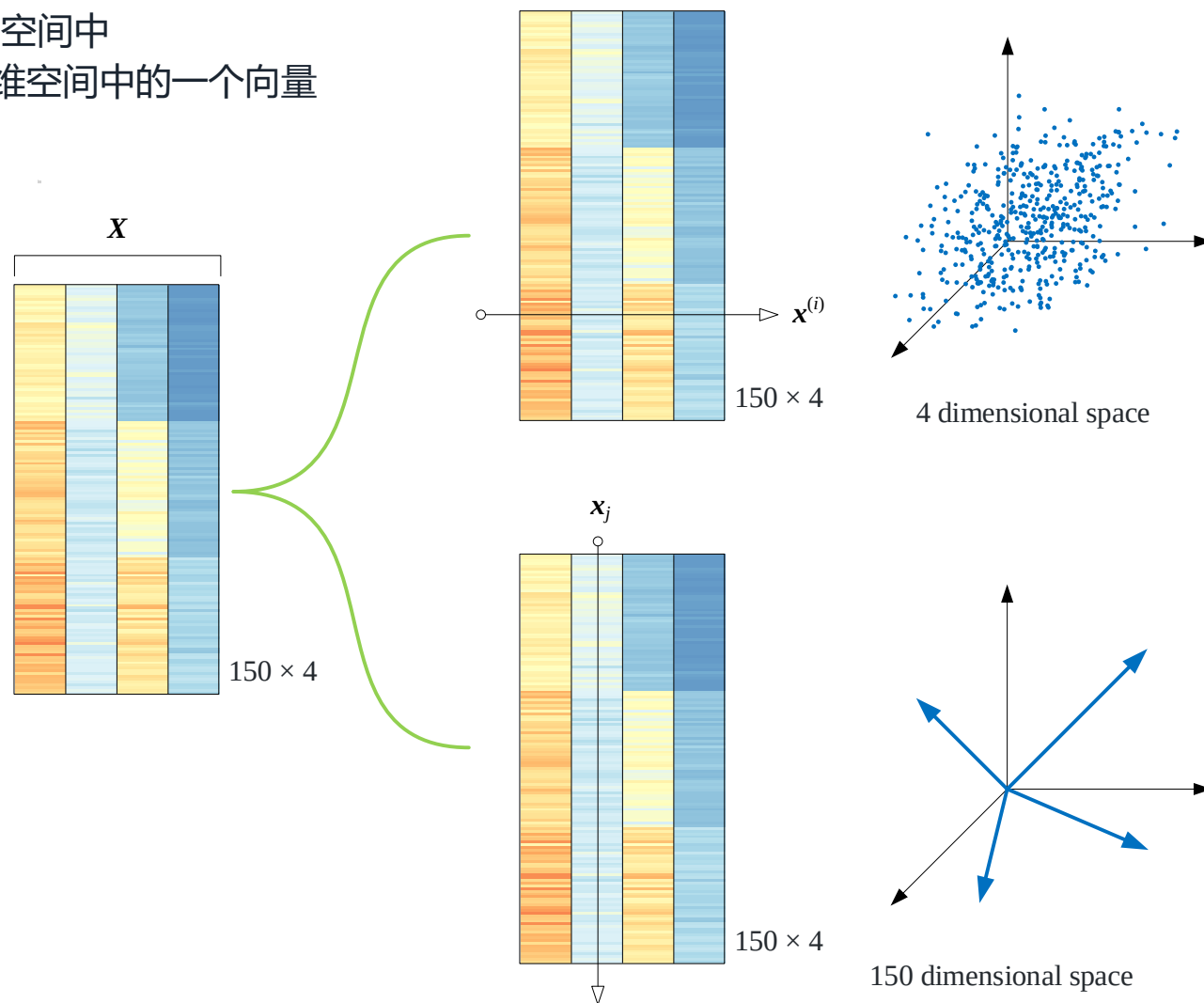
从标量到矩阵：张量的阶

0阶张量 = 标量, 1阶张量 = 向量, 2阶张量 = 矩阵

- 张量 (tensor) 是数学中的多维数组, 用来描述多种维度上的数值数据; 根据阶 (order) 的不同, 能表达不同复杂程度的信息
- 3阶及以上的张量统称为多维数组, 比如彩色图像 (高度×宽度×颜色通道) 就是一个3阶张量
- 一个 D 维空间可记作 $\mathbb{R}^D$: 二维平面是 $\mathbb{R}^2$, 鸢尾花4个特征的空间是 $\mathbb{R}^4$, 150个样本的空间是 $\mathbb{R}^{150}$

图14 • 从行、列视角看鸢尾花数据

- 按行看：每一行 $\mathbf{x}^{(i)}$ 是4个特征组成的点，落在4维空间中
- 按列看：每一列 $\mathbf{x}_j$ 是150个样本的取值，构成150维空间中的一个向量



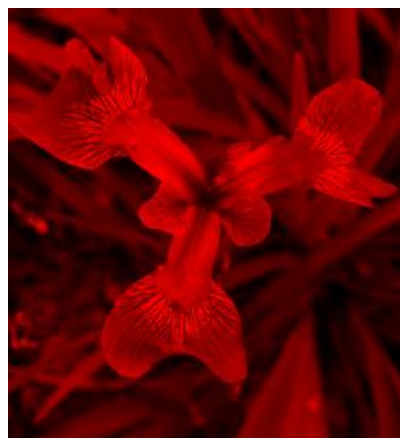
从像素到照片：不同阶数的张量

RGB颜色 $\mathbf{c} = [R, G, B]^T$ 是1阶张量

- 0阶张量的例子：黑白照片中一个像素的灰度值，比如0.5
- 1阶张量的例子：一个RGB颜色，可以用列向量表示为 $[R, G, B]^T$
- 2阶张量的例子：一张黑白照片，两个维度是高度 H 和宽度 W ，记作 $\mathbb{R}^{H \times W}$ ；一个2行3列的矩阵同样是2阶张量

图15 • 彩色照片分解为R、G、B三个通道

- 彩色照片包含R、G、B三个通道，是一个3阶张量，记作 $\mathbb{R}^{H \times W \times C}$
- 每个颜色切片都是独立的2阶张量；视频再加时间（帧）维度，则是4阶张量



张量：现代机器学习的核心工具

标量 $\rightarrow$ 向量 $\rightarrow$ 矩阵 $\rightarrow$ 张量：数据表示能力层层递进

- 神经网络的权重矩阵、图像的像素通道、自然语言处理中的词嵌入向量，都是以张量的形式存在的
- 通过张量，矩阵乘法、卷积等运算得以在GPU等硬件上高效并行计算，这是深度学习能处理大规模数据的关键
- 无论是计算机视觉、自然语言处理还是时间序列分析，张量都是现代人工智能系统不可或缺的基础工具

本节关键术语

矩阵 *Matrix* — 按行、列排列的二维数组

行索引、列索引 *Row/Column Index* — 行索引*i*从上到下、列索引*k*从左到右，均从1开始

分块矩阵 *Block Matrix* — 把大矩阵按行、列划分成若干子矩阵（块）

主对角元、主对角线 *Diagonal Element / Line* — 行、列索引相同的元素及其连线

标量乘法 *Scalar Multiplication* — 标量 k 与矩阵 A 的乘积 kA

矩阵的形状 *Shape ($m \times p$)* — m 为行数， p 为列数，记作 $A_{m \times p}$

行向量、列向量 *Row/Column Vector* — 矩阵的特殊形状：一行多列或多行一列

矩阵元素 *Entry / Component* — 记作 $a_{i,k}$ ， i 为行索引， k 为列索引

零矩阵 *Zero Matrix* — 元素全为0的矩阵，记作 O

张量 *Tensor* — 多维数组；标量为0阶、向量为1阶、矩阵为2阶

配套代码与延伸阅读

代码1 • LA_02_01_01.ipynb

用NumPy创建矩阵、提取行/列向量与元素

延伸阅读 • LA_02_01_02.ipynb

矩阵加减、标量乘法的NumPy实现，建议自学

推荐阅读 • 《矩阵力量》 《编程不难》

本节配图素材来源，深入理解矩阵与照片、数据的关系

开源仓库

全书代码与图像素材：<https://github.com/Visualize-ML>

习题 Q1 – Q10

Q1 学习使用 `seaborn.heatmap()` 绘制热图。

Q3 如何用 NumPy 二维数组构造 Pandas DataFrame?

Q5 请提取 Q4 数组的第1行、第5行、最后一行（分别用两种索引方式，比较结果维度）；第1列、第5列、最后一列；第1、2、3、8行同列位置的元素；以及所有主对角线元素。

Q7 自学 NumPy 的广播机制（broadcasting）。

Q9 请学习使用以下几个 NumPy 函数，并解释每个函数的作用：`flatten`、`column_stack`、`row_stack`、`flip`、`fliplr`、`flipud`、`reshape`。

Q2 什么是 Pandas 数据帧（DataFrame）？和 NumPy 数组有什么区别联系？

Q4 用随机数发生器生成一个8行、18列的数组，用 `seaborn.heatmap()` 对其可视化。

Q6 用随机数发生器生成两个3行、8列的数组，并求解两者的和、差。

Q8 用随机数发生器生成一个8行、8列的数组，计算每列的最大值、最小值、平均值。

Q10 请自学矩阵逐项积（Hadamard product, element-wise product）。

习题 Q11 – Q20

Q11 用 NumPy 创建一个 4×4 矩阵A，编写代码验证A的主对角线元素是否等于 `numpy.diag(A)` 的返回结果。

Q13 编写函数 `make_block_cols(A)`，将任意矩阵A按列拆分为一组列向量，并以列表形式返回。

Q15 编写函数 `is_square(A)`，判断输入矩阵是否为方阵，并用随机生成的矩阵测试该函数。

Q17 用 NumPy 生成三个形状相同的矩阵A、B、C，编写代码验证矩阵加法结合律 $(A+B)+C = A+(B+C)$ 。

Q19 用 `seaborn.heatmap()` 分别可视化矩阵A、标量乘法结果 $2A$ 、 $-1A$ ，观察颜色深浅的变化规律。

Q12 编写函数 `make_block_rows(A)`，将任意矩阵A按行拆分为一组行向量，并以列表形式返回。

Q14 用 NumPy 生成一个 6×6 的随机整数矩阵，编写代码提取其主对角线以外的所有元素（提示：可用 `numpy.eye()` 辅助构造掩码）。

Q16 用 NumPy 分别生成两个形状相同的 5×5 矩阵A、B，编写代码验证矩阵加法交换律 $A+B=B+A$ 是否成立。

Q18 编写函数 `scalar_multiply(A, k)`，手动实现矩阵的标量乘法（不调用 NumPy 运算符 `*`），并与 $A*k$ 的结果对比验证。

Q20 编写代码，用 `numpy.diag()` 根据一维数组 `[1,2,3,4,5]` 创建一个 5×5 的对角方阵，并用热图可视化。

习题 Q21 – Q30

Q21 用 NumPy 生成一个 10×3 的随机矩阵，模拟10个样本、3个特征的数据集，分别按行、按列提取第1个样本与第1个特征。

Q23 用 NumPy 读取一张彩色图片（或用随机数模拟一个 $H \times W \times 3$ 数组），提取并分别可视化其R、G、B三个通道（仿图15）。

Q25 用 Pandas 读取鸢尾花数据集，转换为 NumPy 二维数组后，编写代码提取山鸢尾（Setosa）品种对应的所有行向量。

Q27 用 NumPy 生成一个随机矩阵，编写代码统计其中大于矩阵平均值的元素个数，并将这些位置在热图中高亮标注。

Q29 编写函数 `block_col_matrix(vectors)`，输入一组形状相同的列向量，将它们重新拼接为一个矩阵（提示：可用 `numpy.hstack()`）。

Q22 编写函数 `tensor_order(x)`，输入一个 NumPy 数组，返回其阶数（即 `x.ndim`），并用标量、向量、矩阵、三维数组分别测试。

Q24 编写代码，验证将一张 $H \times W \times 3$ 的彩色图像数组 `reshape` 为一维数组后再 `reshape` 回原形状，图像内容是否保持不变。

Q26 编写函数 `matrix_summary(A)`，输出矩阵A的形状、主对角线元素之和（迹，`trace`）、所有元素之和。

Q28 编写函数 `block_row_matrix(vectors)`，输入一组形状相同的行向量，将它们重新拼接为一个矩阵（提示：可用 `numpy.vstack()`）。

Q30 综合运用本节知识，编写小程序：输入任意矩阵，依次输出其形状、行向量列表、列向量列表、主对角线元素，以及该矩阵乘以标量2的结果。

10 个可深入探讨的 Prompt (1/2)

- 1 请解释为什么矩阵的形状要“先行后列”标记，这种约定与矩阵的“生长方向”有什么关系？
- 2 请用生活中的例子解释“分块矩阵”思想，比如把一份Excel表格按行、按列拆分成若干小表格。
- 3 请证明：任意矩阵与其形状相同的零矩阵相加，结果仍是原矩阵；这一性质在编程中有什么实际用途？
- 4 为什么 NumPy 的索引从0开始，而数学中矩阵的行、列索引习惯从1开始？这种差异容易在实践中导致哪些常见错误？
- 5 请解释主对角线在矩阵运算（如求迹、判断对称矩阵）中的重要性。

10 个可深入探讨的 Prompt (2/2)

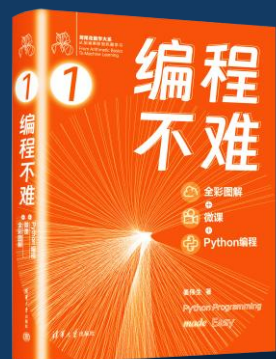
- 6 请设计一个 Python 小实验，验证矩阵加法交换律、结合律对随机生成的矩阵均成立。
- 7 请解释标量乘法 $k=0$ 时结果为零矩阵的几何意义：当我们把一个向量或矩阵乘以0时，实际发生了什么？
- 8 请解释张量的“阶”如何随数据复杂度增加而增加，从标量到向量、矩阵，再到彩色图像、视频。
- 9 神经网络中的权重矩阵、图像的像素通道、词嵌入向量都是张量，请举一个具体机器学习模型的例子说明张量的作用。
- 10 如果要把一个4阶张量（比如视频数据）可视化，你会用什么思路？请给出至少两种方案。

数学

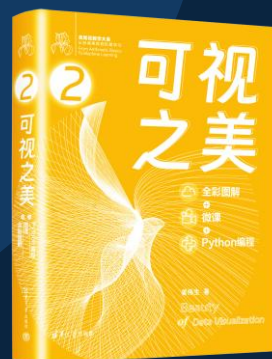
数学基础

线性代数

概率统计

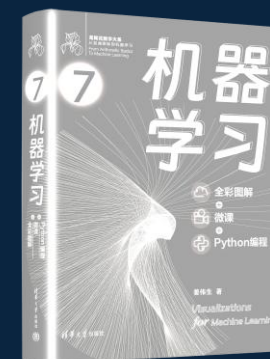
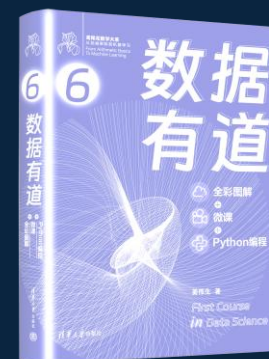
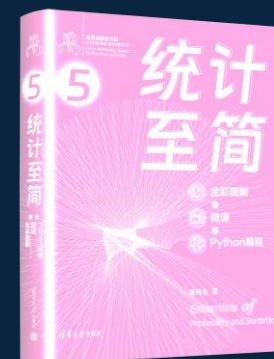
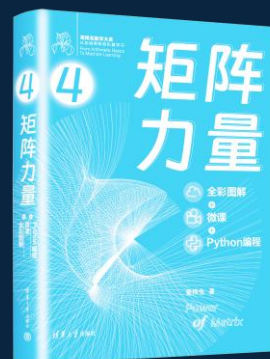
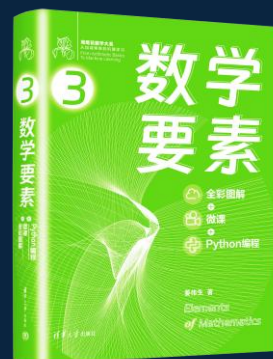


Python编程



数据可视化

工具



数据分析
图和网络

回归、降维
分类、聚类

实践

